

PHASE 1A: EVENT TICKETING PLATFORM

Sections 6, 7, 8 & 9 — Infrastructure Implementation Guides

Redis • RabbitMQ • Stripe • Spring State Machine

This document covers the four core infrastructure implementation guides for Phase 1A. Each section explains **what to build**, **why each design decision was made**, and **exactly how the pieces connect** to the rest of the system. Code blocks have been converted to structured prose — class names, method signatures, key patterns, and configuration names are preserved inline throughout.

SECTION 6

Redis Implementation Guide

Distributed locking, event caching, inventory tracking, cart storage, waitlist management, and reservation expiry

Redis serves **five distinct purposes** in this system — each with its own data type, key pattern, and TTL strategy. Understanding the purpose behind each key prevents misuse and makes debugging straightforward.

6.1 Complete Key Reference

Every Redis key used by the application is listed below. Use this as a lookup table during development and debugging.

Purpose	Key Pattern	Data Type	TTL	Notes
Distributed lock	<code>seat:lock:tier:{tierId}:user:{userId}</code>	String	300s	Value = UUID (lock owner ID). Acquired atomically with SET NX EX. Released via Lua script that checks ownership before deleting.
Cart data	<code>cart:{userId}</code>	Hash	300s	Fields: bookingId, tierId, quantity, eventId. Expires with the reservation. Cleared on checkout or expiry.
Event cache	<code>event:{id}</code>	String (JSON)	600s	Full EventResponse JSON. Cache-aside pattern: miss → DB → store. Evicted on updateEvent and publishEvent.
Event list cache	<code>events:list:{filterHash}</code>	String (JSON)	300s	Paginated list results keyed by a hash of the filter params. Evicted (allEntries=true) on any event update or publish.
Tier availability	<code>inventory:tier:{tierId}:available</code>	String (int)	No TTL	Integer count refreshed from DB on application startup. Decrement on reserveSeat(), increment on releaseSeat().

Purpose	Key Pattern	Data Type	TTL	Notes
Waitlist	<code>waitlist:{eventId}:{tierId}</code>	Sorted Set	No TTL	Score = Unix timestamp (ms) of join time. Member = userId. ZPOPMIN retrieves earliest waiter (FIFO). Popped member is notified via RabbitMQ.
Reservation expiry marker	<code>reservation:expiry:{bookingId}</code>	String	300s	Set when a RESERVED booking is created. Detected by ReservationExpirationJob which runs every 30 seconds. Drives the TIMER_EXPIRED state machine event.

6.2 Data Type Decisions — Why Each Type Was Chosen

Redis offers multiple data structures. Each was chosen for a specific capability required by the use case:

Use Case	Redis Type	Why This Type
Distributed lock	String	SET NX EX is a single atomic command that sets a key only if it does not exist and sets an expiry simultaneously. No other data type offers this atomicity guarantee.
Inventory count	String (int)	Supports INCR and DECR — atomic integer operations that prevent race conditions without external locking.
Cart data	Hash	Groups multiple fields (bookingId, tierId, quantity) under one key. Allows reading individual fields without deserializing the whole structure.
Event cache	String (JSON)	Full object serialized to JSON. Simple cache-aside — get, set, delete. No field-level access needed.
Waitlist	Sorted Set	Scores enable ordered retrieval (FIFO by join timestamp). ZPOPMIN atomically removes and returns the lowest-score member — the earliest waiter.
Expiry marker	String	Only needs to exist (not hold a value). SET with EX gives a self-cleaning key that the expiry job can detect by its presence.

6.3 Distributed Lock — Acquisition and Release

acquireLock() — The Safe Atomic Pattern

The method: `acquireLock(String lockKey, String lockValue, long ttlSeconds)`

Critical Rule: Never use SETNX followed by a separate EXPIRE call. If the process crashes between these two commands, the key is set with no expiry and the lock is held forever. The SET key value NX EX ttl form is a single atomic command — it either sets the key with the TTL or does nothing.

- Calls `redisTemplate.opsForValue().setIfAbsent(lockKey, lockValue, Duration.ofSeconds(ttlSeconds))`, which maps to the atomic SET key value NX EX ttl command.
- The `lockValue` is a `UUID.randomUUID().toString()` generated by the caller. This UUID is the lock owner identity — it is the only value that can release this lock.
- Returns `true` if the lock was acquired (key did not exist and was set), `false` if the lock is already held by another request. The caller should throw `LockAcquisitionException` on `false`.

releaseLock() — Ownership-Checked Lua Script Release

The method: `releaseLock(String lockKey, String lockValue)`

⚠ WARNING: Never use a plain DEL command to release a lock. If your thread's TTL expired and another thread acquired the same lock, DEL would delete their lock — not yours. The Lua script prevents this.

The release uses a Lua script executed atomically by Redis. The script logic in plain English:

1. Read the current value of the lock key from Redis.
 2. Compare it to the `lockValue` argument (the UUID this thread used when acquiring).
 3. If they match: this thread still owns the lock → call DEL and return 1 (success).
 4. If they do not match: another thread now owns the lock → return 0 (do not delete, no-op).
- Execute via `redisTemplate.execute(new DefaultRedisScript<>(luaScript, Long.class), Collections.singletonList(lockKey), lockValue)`.
 - Returns `true` if the lock was successfully released (owned and deleted), `false` if the lock was not owned by this thread (already expired or taken).

executeWithLock() — Safe Convenience Wrapper

The method: `executeWithLock(String lockKey, long ttlSeconds, Supplier<T> operation)`

- Generates a `UUID` as the lock value, then calls `acquireLock()`. Throws `LockAcquisitionException` if the lock cannot be acquired.

- Executes the `Supplier<T>` operation inside a `try` block.
- Calls `releaseLock()` in the `finally` block — guaranteeing release even if the operation throws an exception.
- This is the recommended entry point in `BookingService.reserveTickets()`. All callers should use this wrapper rather than calling `acquireLock()` and `releaseLock()` directly.

✓ **TIP:** Lock key design: use `tier:{tierId}:user:{userId}` to prevent the same user from double-clicking "Reserve" simultaneously. The tier-only key would block all users from that tier during one person's reservation — use the combined key to scope the lock narrowly.

6.4 Inventory Service — Redis-Backed Availability

The `InventoryService` keeps ticket availability counts in Redis for fast reads without hitting the database on every request. The DB remains the source of truth — Redis is the fast cache layer.

Startup Warming

On application startup (`@PostConstruct` or `ApplicationReadyEvent`), the service reads `available_count` for every `TicketTier` from the database and writes each value into Redis using `inventory:tier:{tierId}:available` keys. This seeds the cache before any requests arrive.

Core Methods

- `getAvailableCount(Long tierId)` — Reads from `inventory:tier:{tierId}:available` in Redis. On a cache miss (key absent), falls back to the DB and re-seeds the key. Returns the current integer count.
- `reserveSeat(Long tierId, Long userId, int quantity)` — Called while the distributed lock is held. Uses `redisTemplate.opsForValue().decrement(key, quantity)` for atomic decrement. Returns `false` (and does not decrement) if the resulting count would go below zero.
- `releaseSeat(Long tierId, int quantity)` — Called by `ReleaseSeatsAction` on expiry or cancellation. Uses `redisTemplate.opsForValue().increment(key, quantity)` for atomic increment. After incrementing, also calls `waitlistService.notifyNextInWaitlist()`.
- `commitReservation(Long tierId, int quantity)` — No-op in this architecture because the DB `available_count` column is decremented directly when the Booking is persisted. If you switch to a DB-as-write-behind strategy this method becomes the persistence point.

6.5 Cache Invalidation Strategy

Spring Cache annotations (`@Cacheable`, `@CacheEvict`, `@Caching`) are used throughout `EventService`. The `RedisCacheManager` is configured in `RedisConfig.java` with a default 10-minute TTL and per-cache overrides.

Cache Names and TTLs

Cache Name	TTL	Eviction Triggers
<code>EVENT_CACHE</code>	10 min	Evicted by key (event ID) on <code>updateEvent()</code> and <code>publishEvent()</code> . Single event cache.
<code>EVENT_LIST_CACHE</code>	5 min	Evicted with <code>allEntries=true</code> on any event update or publish — all filter combinations cleared at once.
<code>TIER_AVAILABILITY_CACHE</code>	No TTL (manual)	Managed directly by <code>InventoryService</code> via <code>RedisTemplate</code> , not Spring Cache annotations.

Eviction Rules — Service Layer Annotations

- `getEventById(Long id)` — annotated `@Cacheable(value="EVENT_CACHE", key="#id")`. On hit: returns cached `EventResponse` without touching the DB. On miss: fetches from DB, populates cache, returns result.
- `updateEvent(Long eventId, ...)` — annotated `@CacheEvict(value={"EVENT_CACHE", "EVENT_LIST_CACHE"}, key="#eventId")`. Evicts the specific event entry and all list cache entries simultaneously.
- `publishEvent(Long eventId)` — annotated `@Caching(evict={@CacheEvict(value="EVENT_CACHE", key="#eventId"), @CacheEvict(value="EVENT_LIST_CACHE", allEntries=true)})`. Publishes an event affects every filtered list, so `allEntries=true` wipes the entire list cache rather than guessing which filter combinations are affected.

Why `allEntries=true` on lists? Event list results depend on many filter combinations (category, city, date range, query string). It is impossible to know which cached filter combinations are affected by a single event change. Wiping all list caches on any mutation is safe and simple — list data is cheap to regenerate and has a short TTL anyway.

6.6 Waitlist — Redis Sorted Set Pattern

The waitlist uses a Redis Sorted Set at key `waitlist:{eventId}:{tierId}`. The score is the Unix timestamp in milliseconds at the time the user joined. Lower score = joined earlier = served first (FIFO).

- `joinWaitlist(Long eventId, Long tierId, Long userId)` — Calls `redisTemplate.opsForZSet().add(key, userId.toString(), System.currentTimeMillis())`. If the `userId` is already in the set, the score is updated (re-join refreshes position, which is acceptable).

- `notifyNextInWaitlist(Long eventId, Long tierId)` — Calls `redisTemplate.opsForZSet().popMin(key, 1)` to atomically remove and return the member with the lowest score. If the result is non-empty, publishes a `WaitlistAvailableEvent` to RabbitMQ containing `userId`, `eventId`, and `tierId`. The notification listener then emails the user.
- Called by `releaseSeat()` — so both manual cancellations and timer-expired expirations automatically trigger waitlist notification.

✓ **TIP:** One notification per seat release: `popMin` fetches exactly one user. If they do not claim the seat within 15 minutes (a separate business rule you can add in Phase 1B), the next person in the waitlist could be notified again.

6.7 RedisConfig.java — Bean Definitions

All Redis infrastructure is configured in `common/config/RedisConfig.java`. The following beans must be declared:

- `RedisTemplate<String, Object>` — configured with `StringRedisSerializer` for keys and `Jackson2JsonRedisSerializer` for values. This allows storing Java objects as JSON strings.
- `RedisTemplate<String, String>` — used specifically by `DistributedLockService` where values are UUID strings that do not need JSON serialization.
- `RedisCacheManager` — built from `RedisCacheConfiguration.defaultCacheConfig()` with a default TTL of 10 minutes, key serializer set to `StringRedisSerializer`, and value serializer set to `Jackson2JsonRedisSerializer`. Per-cache TTL overrides are applied via `withInitialCacheConfigurations(Map<String, RedisCacheConfiguration>)`.
- Cache name constants class: define `EVENT_CACHE`, `EVENT_LIST_CACHE`, and `TIER_AVAILABILITY_CACHE` as public static final String fields — used by `@Cacheable` and `@CacheEvict` annotations to avoid hardcoded strings.

6.8 Debugging Redis in Development

Use `redis-commander` (included in `docker-compose.yml` at port `8081`) to inspect all live keys. Key things to check during development:

- **Verify lock keys:** Search for `seat:lock:*`. If a lock key exists with no TTL, the lock was created without `NX EX` — this is a bug in the acquisition code.
- **Verify availability counts:** Search for `inventory:tier:*` and check that the values match `available_count` in the DB after any test reservation.
- **Verify cart expiry:** Search for `cart:*` and check the TTL. It should be ≤ 300 seconds. If the TTL is missing, the cart will never expire.
- **Verify waitlist:** Search for `waitlist:*` and use `ZRANGE` with `WITHSCORES` to view the queued users and their join timestamps.

SECTION 7

RabbitMQ Implementation Guide

Exchange topology, queue declarations, dead letter queues, message serialization, and consumer error handling

RabbitMQ decouples post-payment processing from the critical path. Once a booking is confirmed, a `BookingConfirmedEvent` is published to RabbitMQ and the API returns immediately. Email sending and QR code generation happen asynchronously — **a slow email provider cannot delay the checkout response.**

7.1 Exchange and Queue Topology

The RabbitMQ topology is defined in `common/config/RabbitMQConfig.java` and consists of two exchanges and three primary queues, each with a corresponding Dead Letter Queue (DLQ).

Exchanges

Exchange Name	Type	Purpose
<code>booking.exchange</code>	Topic	Routes booking lifecycle events. Topic type allows pattern-based routing — future routing keys like <code>booking.expired</code> or <code>booking.cancelled</code> can be added without changing the exchange.
<code>notification.exchange</code>	Direct	Routes notification events (email sends, waitlist alerts). Direct type for simple exact-match routing to specific queues.

Queues, Routing Keys, and DLQ Bindings

Queue	Routing Key	Exchange	Msg TTL	DLQ Name
<code>booking.confirmation.queue</code>	<code>booking.confirmed</code>	<code>booking.exchange</code>	24 hr	<code>booking.confirmation.dlq</code>
<code>email.notification.queue</code>	<code>email.send</code>	<code>notification.exchange</code>	24 hr	<code>email.notification.dlq</code>
<code>ticket.generation.queue</code>	<code>ticket.generate</code>	<code>booking.exchange</code>	24 hr	<code>ticket.generation.dlq</code>

Why 24-hour message TTL? If all consumer instances are down (e.g. a deployment), messages accumulate in the queue. A 24-hour TTL prevents unbounded queue growth — if a message cannot be delivered within 24 hours it routes to the DLQ for manual inspection rather than being lost silently.

7.2 RabbitMQConfig.java — Bean Declarations

All constants and Spring beans are declared in `RabbitMQConfig.java` annotated with `@Configuration`. Using public static final String constants (instead of hardcoded strings) means a typo anywhere causes a compilation error, not a silent routing failure.

String Constants (public static final)

- **Exchange names:** `BOOKING_EXCHANGE = "booking.exchange"`, `NOTIFICATION_EXCHANGE = "notification.exchange"`.
- **Queue names:** `BOOKING_CONFIRMATION_QUEUE`, `EMAIL_NOTIFICATION_QUEUE`, `TICKET_GENERATION_QUEUE`.
- **DLQ names:** `BOOKING_CONFIRMATION_DLQ` (and equivalents for email and ticket generation).
- **Routing keys:** `BOOKING_CONFIRMED_KEY = "booking.confirmed"`, `EMAIL_SEND_KEY = "email.send"`, `TICKET_GENERATE_KEY = "ticket.generate"`.

@Bean Methods

- **bookingExchange():** Returns `new TopicExchange(BOOKING_EXCHANGE, true, false)`. The second argument `true` makes it durable (survives broker restart). The third argument `false` means it is not auto-deleted when no queues are bound.
- **notificationExchange():** Returns `new DirectExchange(NOTIFICATION_EXCHANGE, true, false)`.
- **bookingConfirmationQueue():** Returns a durable queue built with `QueueBuilder.durable(BOOKING_CONFIRMATION_QUEUE)`. Three arguments are added:
 - `x-dead-letter-exchange: ""` (the default exchange — routes to queues by name).
 - `x-dead-letter-routing-key: BOOKING_CONFIRMATION_DLQ` (DLQ queue name as the routing key).
 - `x-message-ttl: 86400000` (24 hours in milliseconds).
- **bookingConfirmationDLQ():** Returns `QueueBuilder.durable(BOOKING_CONFIRMATION_DLQ).build()`. A plain durable queue — no DLQ of its own, no TTL. Messages sit here until a developer inspects and re-queues or discards them.
- **bookingConfirmationBinding():** Returns `BindingBuilder.bind(bookingConfirmationQueue()).to(bookingExchange()).with(BOOKING_CONFIRMED_KEY)`.
- **jackson2JsonMessageConverter():** Returns `new Jackson2JsonMessageConverter()`. Registered as the message converter so that Java POJOs are automatically serialized to JSON when publishing and deserialized back when consuming. No manual JSON handling needed in publishers or listeners.

- **rabbitTemplate():** Returns a `RabbitTemplate` with `setMessageConverter(jackson2JsonMessageConverter())` applied. This is the bean injected into `BookingEventPublisher`.

7.3 BookingEventPublisher — Sending Messages

The `BookingEventPublisher` component wraps `RabbitTemplate` with three named publish methods, one per message type. Callers never interact with `RabbitTemplate` directly — they call the named method with a typed POJO, and Jackson serializes it to JSON automatically.

- `publishBookingConfirmation(BookingConfirmedEvent event)` — calls `rabbitTemplate.convertAndSend(BOOKING_EXCHANGE, BOOKING_CONFIRMED_KEY, event)`. Called from `StripeWebhookController.handlePaymentSuccess()` after the booking transitions to CONFIRMED.
- `publishEmailNotification(EmailNotificationEvent event)` — calls `rabbitTemplate.convertAndSend(NOTIFICATION_EXCHANGE, EMAIL_SEND_KEY, event)`. Used for general transactional emails (e.g. account registration, password reset).
- `publishTicketGeneration(TicketGenerationEvent event)` — calls `rabbitTemplate.convertAndSend(BOOKING_EXCHANGE, TICKET_GENERATE_KEY, event)`. Triggers batch QR code generation for large group bookings where inline generation would be too slow.

Message POJO Requirements

Each message class (`BookingConfirmedEvent`, `EmailNotificationEvent`, `TicketGenerationEvent`, `WaitlistAvailableEvent`) must:

- Have a no-argument constructor (required by Jackson for deserialization on the consumer side).
- Be annotated with `@Data` and `@Builder` (Lombok) for construction and inspection.
- Contain all fields that the consumer needs — avoid lazy loading or DB calls inside listeners; the message payload should be self-contained.

7.4 Consumer — Manual Acknowledgement and Error Handling

The `BookingNotificationListener` (and all other listeners) use **manual acknowledgement** rather than Spring's default auto-ack. This ensures a message is only removed from the queue after it has been fully processed.

Why Manual Acknowledgement?

- With auto-ack (the default), RabbitMQ removes the message from the queue as soon as it is delivered to the consumer — even if the consumer crashes mid-processing. The message is lost.

- With manual ack, the message remains in the queue (as "unacknowledged") until the consumer explicitly calls `basicAck()`. If the consumer crashes, RabbitMQ requeues the message automatically.

Handler Method Signature and Flow

The `@RabbitListener` method accepts three parameters:

- `BookingConfirmedEvent event` — the deserialized message payload (Jackson handles this automatically via `Jackson2JsonMessageConverter`).
- `@Header(AmqpHeaders.DELIVERY_TAG) long deliveryTag` — a unique identifier for this message delivery, required for explicit ack/nack calls.
- `Channel channel` — the AMQP channel, used to call `basicAck()` or `basicNack()`.

Processing flow inside the handler:

5. **Log entry:** Log `"Processing booking confirmation: bookingId={}"` at INFO level using `@Slf4j`.
6. **Process:** Fetch any additional details needed, generate PDF (optional for MVP), call `emailService.sendBookingConfirmation(event)`.
7. **Acknowledge:** On success, call `channel.basicAck(deliveryTag, false)`. The second argument `false` means "acknowledge only this message" (not all unacknowledged messages up to this one).
8. **Handle failure:** In the `catch(Exception e)` block, log the error at ERROR level, then call `channel.basicNack(deliveryTag, false, false)`. The third argument `false` means "do not requeue" — the message routes to the DLQ instead of being retried immediately (which could cause infinite retry loops on persistent errors).

DLQ monitoring: Subscribe to the DLQ in Mailhog or the RabbitMQ Management UI (port 15672). Any message that lands in the DLQ is a processing failure that needs investigation. In production, set up an alert on DLQ depth.

7.5 Verifying the Topology

After the application starts, open the RabbitMQ Management UI at `http://localhost:15672` (default credentials: guest/guest, or the values set in `docker-compose.yml`). Verify:

- Click the **Exchanges** tab: `booking.exchange` and `notification.exchange` are listed as durable.
- Click the **Queues** tab: all three primary queues and all three DLQs are listed as durable. The `x-dead-letter-exchange` and `x-dead-letter-routing-key` arguments should be visible in each queue's detail view.
- Click **Bindings** on each queue: the routing key binding to the correct exchange is listed.

✓ **TIP:** Use the Management UI's "Publish message" tool to manually inject test messages into queues during development — much faster than triggering a full payment flow every time you want to test a listener.

SECTION 8

Stripe Implementation Guide

Checkout session creation, webhook verification, idempotency, refund processing, and environment configuration

The Stripe integration follows the **Checkout + Webhook pattern**: the backend creates a Checkout Session and returns the hosted Stripe URL; payment happens on Stripe's page; a signed webhook event notifies the backend of the outcome. **The webhook is the only reliable signal** — never trust a redirect URL parameter to confirm payment.

8.1 Environment Configuration

All Stripe configuration is read from environment variables — never hardcoded. Add the following to `application.yml`:

application.yml Key	Env Var	Description
<code>stripe.secret-key</code>	<code>STRIPE_SECRET_KEY</code>	Your <code>sk_test_...</code> or <code>sk_live_...</code> key. Never committed to source control.
<code>stripe.webhook-secret</code>	<code>STRIPE_WEBHOOK_SECRET</code>	Signing secret from Stripe CLI (<code>whsec_...</code>) or Stripe Dashboard. Required for signature verification.
<code>stripe.success-url</code>	<code>FRONTEND_URL + path</code>	Redirect URL after successful payment. Must include <code>{bookingId}</code> and <code>{CHECKOUT_SESSION_ID}</code> placeholders.
<code>stripe.cancel-url</code>	<code>FRONTEND_URL + /cart</code>	Redirect URL if the user abandons the Stripe Checkout page.

StripeConfig.java: Annotated `@Configuration`. Injects `${stripe.secret-key}` via `@Value` and assigns it to `Stripe.apiKey` in a `@PostConstruct` method. This must run before any Stripe SDK call — Spring guarantees `@PostConstruct` runs after all injection is complete.

8.2 createCheckoutSession() — Step-by-Step

Located in `PaymentService.java`. Called from `POST /api/bookings/{id}/checkout`.

Pre-conditions (validate before any Stripe call)

9. **Fetch and validate booking:** Call `bookingRepository.findByIdAndUserId(bookingId, userId)`. Throw `BookingNotFoundException` if not found — prevents users from checking out another user's booking.
10. **Validate state:** Assert `booking.getState() == BookingState.RESERVED`. Throw `InvalidStateTransitionException` with message "Booking must be in RESERVED state to checkout" if not.
11. **Transition state:** Call `bookingStateMachineService.sendEvent(bookingId, BookingEvent.PROCEED_TO_CHECKOUT)`. This moves the booking to `PAYMENT_PENDING` before the Stripe session is created — if the session creation fails, a subsequent retry will find the booking in `PAYMENT_PENDING` and skip the state transition.

Build the Stripe Session

12. **Build line items:** Call `buildLineItems(booking)` (a private helper). Iterates `booking.getTickets()` and creates one `SessionCreateParams.LineItem` per ticket. Each line item carries `quantity: 1L`, `currency: "usd"`, `unitAmount: (long) (tier.getPrice().doubleValue() * 100)` (Stripe always uses cents), and `productName: tierName + " - " + eventTitle`.
13. **Build session params:** Use `SessionCreateParams.builder()` with:
 - `.setMode(SessionCreateParams.Mode.PAYMENT)` — one-time payment (not subscription).
 - `.setCustomerEmail(booking.getUser().getEmail())` — pre-fills email on the Stripe hosted page.
 - `.setSuccessUrl(successUrl.replace("{bookingId}", bookingId.toString()))` — redirect after payment. Stripe also appends `?session_id={CHECKOUT_SESSION_ID}` automatically.
 - `.setCancelUrl(cancelUrl)` — redirect if user clicks Back.
 - `.addAllLineItem(lineItems)` — the ticket list.
 - `.setExpiresAt(Instant.now().plus(30, ChronoUnit.MINUTES).getEpochSecond())` — Stripe session expires after 30 minutes. Aligns with the 5-minute reservation expiry plus a buffer.
 - `.setMetadata(Map.of("bookingId", bookingId.toString(), "userId", userId.toString()))` — critical: the webhook handler reads these values to identify which booking to confirm.
14. **Create session:** Call `Session.create(params)`. This makes an HTTPS call to the Stripe API. Wrap in try-catch for `StripeException` — log and throw a `PaymentProcessingException` if it fails.

Persist and Return

15. **Save session ID on booking:** Call `booking.setStripeSessionId(session.getId())` and `bookingRepository.save(booking)`. This allows the webhook handler to look up the booking by session ID if the metadata is missing.

16. **Create Payment entity:** Build and save a `Payment` with `booking`, `stripeSessionId: session.getId()`, `amount: booking.getTotalAmount()`, `status: PaymentStatus.PENDING`.
17. **Return:** `new CheckoutSessionResponse(session.getUrl())`. The frontend receives this URL and redirects the user to Stripe.

8.3 StripeWebhookController — Webhook Handling

Located at `POST /api/webhooks/stripe`. The endpoint accepts a raw `String` body (not a deserialized object) and the `Stripe-Signature` header. It must be excluded from CSRF protection in `SecurityConfig`.

Step 1 — Signature Verification

- Call `Webhook.constructEvent(payload, sigHeader, webhookSecret)`.
- If `SignatureVerificationException` is thrown: log at WARN level, return `ResponseEntity.badRequest().body("Invalid signature")`. Never process an unsigned event — this would allow anyone on the internet to fake payment confirmations.
- The `@RequestBody String payload` annotation must receive the raw bytes — do not use `@RequestBody` with a deserialized type, as that would break the signature verification (Stripe signs the exact byte sequence of the JSON payload).

⚠ WARNING: Do not read the request body twice. Spring's `HttpServletRequest.getInputStream()` can only be read once. If any filter or interceptor reads the body before this controller, signature verification will fail. Configure Spring Security to not consume the body for this endpoint.

Step 2 — Idempotency Check

- Check `processedEventRepository.existsByStripeEventId(stripeEvent.getId())`.
- If `true`: log at INFO and return `ResponseEntity.ok("Already processed")` immediately. Stripe may deliver the same event more than once (network retries, delivery guarantees). Processing twice would double-confirm a booking, generate duplicate QR codes, and send duplicate emails.
- The `ProcessedStripeEvent` table needs a unique index on `stripe_event_id` at the DB level as a second guard.

Step 3 — Event Routing

- Switch on `stripeEvent.getType()`:

- `checkout.session.completed` → deserialize `Session` via `stripeEvent.getDataObjectDeserializer().getObject().orElseThrow()`. Call `paymentService.handlePaymentSuccess(session)`
- `checkout.session.expired` → deserialize `Session`. Call `paymentService.handlePaymentExpired(session)`

`handlePaymentSuccess(Session session)` steps:

18. Extract `bookingId` from `session.getMetadata().get("bookingId")`.
19. Update `Payment` entity: set `status = PaymentStatus.COMPLETED` and `stripePaymentIntentId = session.getPaymentIntent()`.
20. Send `BookingEvent.PAYMENT_SUCCESS` to the state machine — transitions booking to `CONFIRMED`, fires `GenerateQRCodeAction`.
21. Call `bookingEventPublisher.publishBookingConfirmation(new BookingConfirmedEvent(...))`.
22. Save `ProcessedStripeEvent` with `stripeEvent.getId()`.

`handlePaymentExpired(Session session)` steps:

23. Extract `bookingId` from session metadata.
24. Send `BookingEvent.PAYMENT_FAILURE` to the state machine — transitions booking to `PAYMENT_FAILED` and fires `ReleaseSeatsAction`.
25. Update `Payment` entity `status = PaymentStatus.FAILED`.

Step 4 — Return

- Always return `ResponseEntity.ok("Received")` after processing. Stripe retries the webhook delivery if it receives a non-2xx response — returning 200 even after handling errors prevents infinite retries for errors that are not Stripe's fault.

8.4 `processRefund()` — Refund Processing

Located in `RefundService.java`. Called from `DELETE /api/bookings/{id}` (cancel booking). The refund amount is calculated by `PricingEngine.calculateRefundAmount()` based on how far the event is in the future.

26. **Fetch and validate booking:** Call `bookingRepository.findByIdAndUserId(bookingId, userId)`. Throw `BookingNotFoundException` if not found.
27. **Calculate refund amount:** Call `pricingEngine.calculateRefundAmount(booking, LocalDateTime.now())`. Returns a `BigDecimal` — the amount to refund.
28. **Check eligibility:** If `refundAmount.compareTo(BigDecimal.ZERO) == 0`: throw `RefundNotEligibleException("No refund available within 3 days of event")`.

29. **Fetch payment:** Call `paymentRepository.findByBookingId(bookingId).orElseThrow()`. The `stripe_payment_intent_id` is required for the Stripe refund call.
30. **Call Stripe Refund API:** Build `RefundCreateParams` with:
 - `.setPaymentIntent(payment.getStripePaymentIntentId())` — references the original charge.
 - `.setAmount(refundAmount.multiply(BigDecimal.valueOf(100)).longValue())` — converts decimal dollars to cents.
 - `.setReason(RefundCreateParams.Reason.REQUESTED_BY_CUSTOMER)` — populates the reason field in the Stripe dashboard.
 - Call `com.stripe.model.Refund.create(refundParams)`.
31. **Persist Refund entity:** Build and save a `Refund` with `stripeRefundId`:
`stripeRefund.getId(), amount: refundAmount, status: RefundStatus.APPROVED, processedAt: LocalDateTime.now()`.
32. **Transition state machine:** Send `BookingEvent.APPROVE_REFUND` → booking moves to `REFUND_APPROVED` and `ReleaseSeatsAction` fires.
33. **Release seats:** Call `inventoryService.releaseSeat(tierId, booking.getTickets().size())`. Increments the Redis availability count.
34. **Notify waitlist:** Call `waitlistService.notifyNextInWaitlist(eventId, tierId)`. Pops the next user from the waitlist Sorted Set and publishes a `WaitlistAvailableEvent`.
35. **Return:** `new RefundResponse(refund)`.

Partial refunds: The amount argument to `Stripe Refund.create()` can be less than the original charge. The `PricingEngine` returns 50% of total_amount for cancellations 3–6 days before the event. Stripe handles the partial refund correctly — the remainder stays captured.

8.5 Local Webhook Testing with Stripe CLI

The Stripe CLI is the recommended approach for local webhook testing. It forwards real Stripe webhook events to your local server without needing a public URL.

- **Install:** `brew install stripe/stripe-cli/stripe` (macOS) or download the binary.
- **Login:** `stripe login` — opens a browser to authenticate.
- **Start forwarding:** `stripe listen --forward-to localhost:8080/api/webhooks/stripe`.
- **Copy signing secret:** The CLI prints a `whsec_...` signing secret. Set this as `STRIPE_WEBHOOK_SECRET` in `application-local.yml`.
- **Trigger test events:** `stripe trigger checkout.session.completed` — sends a test event to your listener.

Test Card Numbers (Use in Stripe Checkout test mode only)

Card Number	Scenario	Notes
4242 4242 4242 4242	Payment success	Use any future expiry date, any 3-digit CVC

Card Number	Scenario	Notes
4000 0000 0000 0002	Payment declined	Tests handlePaymentExpired() flow
4000 0025 0000 3155	3D Secure authentication required	Tests the auth redirect flow
4000 0000 0000 9995	Insufficient funds decline	Tests handlePaymentExpired() flow

SECTION 9

State Machine Implementation Guide

BookingStateMachineConfig, stateless-per-request wrapper, all transitions, guards, actions, and test patterns

The Spring State Machine defines and enforces the legal state transitions for every Booking. Without it, a developer could accidentally confirm a booking that was never reserved, or approve a refund on an expired booking. The state machine makes illegal transitions **compile-time safe and runtime-enforced**.

9.1 State and Event Enums

BookingState — All 11 States

State	Meaning
AVAILABLE	Initial state. Tickets exist but no booking has been initiated. This is a virtual state — no Booking entity exists in this state; it represents the absence of a booking.
RESERVED	User added tickets to cart. Seats are locked in Redis. Booking entity created with expires_at = now + 5 minutes. The 5-minute countdown timer starts.
PAYMENT_PENDING	User proceeded to checkout. Stripe Checkout Session created. Booking is awaiting the webhook confirmation from Stripe.
CONFIRMED	Payment received (webhook confirmed). QR codes generated for all tickets. Booking confirmation email sent via RabbitMQ.
ATTENDED	User checked in at the event door. QR code was scanned and verified. Terminal happy-path state.
EXPIRED	Reservation timer ran out (5 minutes elapsed without checkout). Seats automatically released back to inventory. User must start over.
RELEASED	Seats released back to inventory after a PAYMENT_FAILED or EXPIRED transition. Internal cleanup state — not directly shown in the UI.
PAYMENT_FAILED	Stripe reported payment failure or session expiry. Seats released. User sees a payment failure message.
REFUND_REQUESTED	User cancelled a CONFIRMED booking. Refund eligibility checked by PricingEngine. Awaiting processing.
REFUND_APPROVED	Stripe refund processed successfully. Seats released, waitlist notified. Terminal state for cancelled bookings.
REFUND_DENIED	Cancellation requested but refund policy gives 0% refund (< 3 days before event). Booking remains in this state — no seats released.

BookingEvent — 9 State Machine Triggers

Event	Triggered By	Effect
ADD_TO_CART	BookingService.reserveTickets()	Moves AVAILABLE → RESERVED. Guard checks seat availability first.
PROCEED_TO_CHECKOUT	PaymentService.createCheckoutSession()	Moves RESERVED → PAYMENT_PENDING. Prevents second checkout while session is open.
PAYMENT_SUCCESS	StripeWebhookController (checkout.session.completed)	Moves PAYMENT_PENDING → CONFIRMED. Triggers QR generation and confirmation email.
PAYMENT_FAILURE	StripeWebhookController (checkout.session.expired)	Moves PAYMENT_PENDING → PAYMENT_FAILED. Releases seats.
TIMER_EXPIRED	ReservationExpirationJob (@Scheduled)	Moves RESERVED → EXPIRED. Releases seats.
CHECK_IN	Check-in API endpoint (Day 21 feature)	Moves CONFIRMED → ATTENDED.
REQUEST_REFUND	BookingController.cancelBooking()	Moves CONFIRMED → REFUND_REQUESTED.
APPROVE_REFUND	RefundService.processRefund()	Moves REFUND_REQUESTED → REFUND_APPROVED. Releases seats, notifies waitlist.
DENY_REFUND	RefundService.processRefund() (< 3 days rule)	Moves REFUND_REQUESTED → REFUND_DENIED. No seats released.

9.2 Complete Transition Table

Every legal transition in the state machine is listed below. Any transition not listed here is illegal and will cause the state machine to reject the event and throw `InvalidStateTransitionException`.

From State	Event	To State	Guard	Action(s)
AVAILABLE	ADD_TO_CART	RESERVED	SeatsAvailableGuard	StartReservationTimerAction
RESERVED	PROCEED_TO_CHECKOUT	PAYMENT_PENDING	—	—
PAYMENT_PENDING	PAYMENT_SUCCESS	CONFIRMED	—	ConfirmBookingAction GenerateQRCodeAction
CONFIRMED	CHECK_IN	ATTENDED	—	—
RESERVED	TIMER_EXPIRED	EXPIRED	—	ReleaseSeatsAction

From State	Event	To State	Guard	Action(s)
PAYMENT_PENDING	PAYMENT_FAILURE	PAYMENT_FAILED	—	ReleaseSeatsAction
EXPIRED	RELEASE	AVAILABLE	—	—
PAYMENT_FAILED	RELEASE	AVAILABLE	—	—
CONFIRMED	REQUEST_REFUND	REFUND_REQUESTED	—	—
REFUND_REQUESTED	APPROVE_REFUND	REFUND_APPROVED	—	ReleaseSeatsAction
REFUND_REQUESTED	DENY_REFUND	REFUND_DENIED	—	—

9.3 Guards and Actions

Guards — Conditions That Block Transitions

Guards implement `Guard<BookingState, BookingEvent>` and return `true` to allow the transition or `false` to block it. If a guard returns `false`, the state machine rejects the event without throwing an exception by default — your wrapper must check the return value of `sm.sendEvent()` and throw `InvalidStateTransitionException` explicitly if it returns `false`.

- **SeatsAvailableGuard:** Implements `Guard<BookingState, BookingEvent>`. Reads `tierId` from the message header, calls `inventoryService.getAvailableCount(tierId)`, and returns `count > 0`. This is the gate that prevents overselling before the Redis lock is even acquired.
- **BookingOwnedByUserGuard:** Reads `bookingId` and `userId` from message headers. Calls `bookingRepository.existsByIdAndUserId(bookingId, userId)`. Returns `true` only if the requesting user owns the booking. Used on `REQUEST_REFUND` and cancellation transitions.

Actions — Side Effects on Transitions

Actions implement `Action<BookingState, BookingEvent>`. They fire synchronously during the transition. All actions have access to the `StateContext` which carries message headers and the extended state variables — this is how they retrieve `bookingId` and other context values.

Action Class	What It Does
<code>StartReservationTimerAction</code>	Sets <code>booking.expires_at = LocalDateTime.now().plusMinutes(5)</code> and saves the booking. Also writes the reservation:expiry:{bookingId} key to Redis with a 300-second TTL. The <code>ReservationExpirationJob</code> detects this key when it runs every 30 seconds.

Action Class	What It Does
<code>ConfirmBookingAction</code>	Fetches the Payment entity and sets status = <code>PaymentStatus.COMPLETED</code> . Updates <code>stripe_payment_intent_id</code> from the session metadata stored in the extended state. Saves the Payment entity.
<code>GenerateQRCodeAction</code>	Iterates all Ticket entities for the booking. For each ticket, builds a signed JSON payload (ticketId, bookingId, eventId, userId, seatNumber), encodes it as a 200x200 QR code using ZXing <code>QRCodeWriter</code> , converts to Base64, and sets <code>ticket.qrCode = base64String</code> . Saves all tickets.
<code>ReleaseSeatsAction</code>	Reads tierId and quantity from the booking. Calls <code>inventoryService.releaseSeat(tierId, quantity)</code> to increment the Redis availability count. Then calls <code>waitlistService.notifyNextInWaitlist(eventId, tierId)</code> to notify the next user waiting for a seat.

9.4 BookingStateMachineConfig.java — Structure

The config class is annotated `@Configuration`, `@EnableStateMachineFactory`, and `@Slf4j`. It extends `StateMachineConfigurerAdapter<BookingState, BookingEvent>` and overrides three methods:

- **`configure(StateMachineStateConfigurer)`**: Declares the initial state (`BookingState.AVAILABLE`), the end state (`BookingState.ARCHIVED`), and all states via `EnumSet.allOf(BookingState.class)`.
- **`configure(StateMachineTransitionConfigurer)`**: Chains all 11 transitions using the fluent builder pattern. Each transition uses `.withExternal()` (state changes between different states) with `.source()`, `.target()`, `.event()`, optional `.guard()`, and optional `.action()` calls. Transitions are chained with `.and()`.
- **`configure(StateMachineConfigurationConfigurer)`**: Sets `.autoStartup(false)` (the machine is started manually per request in the service wrapper, not globally on startup). Registers a `StateMachineListenerAdapter` that logs every state transition at INFO level using SLF4J.

@EnableStateMachineFactory vs @EnableStateMachine: Use `@EnableStateMachineFactory` (not `@EnableStateMachine`) because the stateless-per-request pattern requires a new `StateMachine` instance per request, retrieved from the factory. `@EnableStateMachine` creates a single shared instance which would have race conditions under concurrent load.

9.5 BookingStateMachineService — Stateless-Per-Request Wrapper

The `BookingStateMachineService` is the only class in the application that calls the Spring State Machine directly. All other services (`BookingService`, `PaymentService`, `RefundService`) call

`bookingStateMachineService.sendEvent(bookingId, event)` — they never touch the state machine factory themselves.

Why Stateless-Per-Request?

- Spring State Machine is not inherently thread-safe when shared. If a single machine instance is reused across concurrent requests, two threads could simultaneously attempt transitions and corrupt each other's state.
- The stateless-per-request pattern solves this: for every `sendEvent()` call, a fresh `StateMachine` instance is obtained from the factory, its state is restored from the database, the event is processed, and the resulting state is persisted back to the database. The machine is then discarded.
- The DB is the authoritative state store. Redis, the machine instance, and all in-memory objects are ephemeral.

sendEvent() — Step-by-Step

36. Get machine from factory: Call

`stateMachineFactory.getStateMachine(bookingId.toString())`. The booking ID is used as the machine ID for logging purposes.

37. Restore current state: Fetch the `Booking` entity from the DB. Call

`sm.getStateMachineAccessor().doWithRegion(access -> access.resetStateMachine(new DefaultStateMachineContext<>(booking.getState(), null, null, null)))`. This forces the machine to the state currently stored in the DB, discarding any prior in-memory state.

38. Start the machine: Call `sm.start()`.

39. Inject context variables: Call

`sm.getExtendedState().getVariables().put("bookingId", bookingId)` and any other context values (tierId, userId, quantity) that actions and guards need during the transition.

40. Build and send the message: Build

`MessageBuilder.withPayload(event).setHeader("bookingId", bookingId).build()`. Call `boolean accepted = sm.sendEvent(message)`

41. Check acceptance: If `!accepted`: throw `InvalidStateTransitionException("Cannot send event " + event + " in state " + booking.getState())`.

42. Persist new state: Read `sm.getState().getId()` and call `booking.setState(newState) + bookingRepository.save(booking)`.

43. Stop the machine: Call `sm.stop()`. Releases resources.

✓ **TIP:** Wrap `sendEvent()` calls in `@Transactional` at the service layer (e.g. `BookingService`, `PaymentService`). This ensures that if the DB save of the new state fails, the whole operation rolls back — you never end up with a state machine that transitioned but a DB that still shows the old state.

9.6 Testing State Transitions

State machine tests live in `BookingStateMachineTest.java`, annotated `@ExtendWith(MockitoExtension.class)`. The service is constructed with `@InjectMocks`. The state machine factory, booking repository, lock service, and inventory service are all mocked with `@Mock`.

Required Test Cases (5 minimum — all must pass)

Test Method Name	What to Assert
<code>transition_availableToReserved_shouldLockAndDecrementInventory</code>	After <code>sendEvent(ADD_TO_CART)</code> : <code>booking.getState() == RESERVED</code> ; <code>verify(inventoryService).reserveSeat(tierId, 1)</code> ; <code>verify(lockService).acquireLock(anyString(), anyString(), eq(300L))</code> .
<code>transition_reservedToExpired_shouldReleaseSeats</code>	After <code>sendEvent(TIMER_EXPIRED)</code> : <code>booking.getState() == EXPIRED</code> ; <code>verify(inventoryService).releaseSeat(tierId, quantity)</code> ; <code>verify(waitlistService).notifyNextInWaitlist(eventId, tierId)</code> .
<code>transition_paymentPendingToConfirmed_shouldGenerateQRcodesForAllTickets</code>	After <code>sendEvent(PAYMENT_SUCCESS)</code> : <code>booking.getState() == CONFIRMED</code> ; all tickets have non-null qrCode values <code>(assertThat(ticket.getQrCode()).isNotNull() for each)</code> .
<code>seatsAvailableGuard_whenNoSeatsAvailable_shouldBlockTransition</code>	Mock <code>inventoryService.getAvailableCount()</code> to return 0. Assert that <code>sendEvent(ADD_TO_CART)</code> throws <code>InvalidStateTransitionException</code> or returns false (no transition occurred).
<code>invalidTransition_confirmedToReserved_shouldThrowException</code>	Given a CONFIRMED booking, calling <code>sendEvent(ADD_TO_CART)</code> must throw <code>InvalidStateTransitionException</code> . The booking state must remain CONFIRMED after the call.

Test Setup Pattern

- Use `@BeforeEach` to set up a default booking in a known state (e.g. RESERVED). Configure mock return values for `bookingRepository.findById()` to return this booking.
- Mock the state machine factory to return a real `StateMachine` instance configured with the actual `BookingStateMachineConfig` — this gives you genuine state machine behaviour rather than mocked transitions.

- Use `@DisplayName` on every test with a human-readable description: `"AVAILABLE → RESERVED: should acquire Redis lock and decrement inventory"`. This makes the test report readable without needing to decode method names.
- Use AssertJ (`assertThat()`) rather than JUnit `assertEquals()` — the fluent API produces more descriptive failure messages.

9.7 Common State Machine Mistakes

Mistake	Correct Approach
Using <code>@EnableStateMachine</code> (singleton) instead of <code>@EnableStateMachineFactory</code>	Use <code>@EnableStateMachineFactory</code> always. The per-request pattern requires a fresh instance per <code>sendEvent()</code> call — a singleton would have concurrent state corruption under load.
Not checking <code>sm.sendEvent()</code> return value	Always check the boolean return. If <code>sendEvent()</code> returns false, the transition was rejected (guard returned false or illegal transition). Throw <code>InvalidStateTransitionException</code> explicitly.
Not calling <code>sm.stop()</code> after use	Call <code>sm.stop()</code> in a finally block. Failure to stop leaks resources and can cause the machine to appear "still running" in logs.
Calling <code>sendEvent()</code> without first restoring state via <code>resetStateMachine()</code>	Always call <code>doWithRegion + resetStateMachine</code> before <code>sm.start()</code> . Without this, the machine starts in its initial state (AVAILABLE) instead of the booking's current DB state.
Putting business logic inside the Config class	Keep <code>BookingStateMachineConfig</code> purely declarative. All logic belongs in the Guard and Action beans — which are separate <code>@Component</code> classes injected into the config.
Not using <code>@Transactional</code> around <code>sendEvent()</code> calls	If the DB save of the new state fails, the action (e.g. QR generation) has already run but the state is not persisted. Wrap the entire <code>sendEvent()</code> + <code>bookingRepository.save()</code> sequence in a <code>@Transactional</code> boundary.